

RQ2: How does Test-Driven Development (TDD) impact software quality?

A Systematic Synthesis of Five Verified 2024 Empirical and Review Studies

Group 3

Faculty of Information Technology

July 2026

Abstract—Test-Driven Development (TDD) is one of the most extensively studied practices in agile software engineering, yet the empirical literature on its effect on software quality remains fragmented, and in several cases directly contradictory. This report synthesizes five independently verified studies published in 2024 – retrieved and cross-checked through Google Scholar, OpenAlex, ResearchGate, Connected Papers, and Litmaps – to answer the research question RQ2: *How does Test-Driven Development (TDD) impact software quality?* The evidence base spans five distinct methodological angles: a mixed-methods practitioner survey on perceived quality gains, a meta-methodological reanalysis of the measurement instruments used in TDD experiments, an exploratory study of generative-AI-assisted TDD, a qualitative enterprise-level comparison of TDD against Behavior-Driven Development (BDD), and a systematic literature review (SLR) connecting TDD practice to test-code smells. Rather than treating these as five independent verdicts on “does TDD work,” this report treats them as five lenses on the same underlying phenomenon, and explicitly synthesizes where they agree, where they disagree, and *why* they disagree. The central and most actionable finding is that TDD’s measured effect on software quality is not merely mixed by chance – it is demonstrably sensitive to how “quality” is operationalized and measured, to the point that reanalysis of a single industrial dataset can produce a statistically significant positive *or* negative TDD effect depending solely on which acceptance-test suite is used to score the code. Layered on top of this measurement-validity concern are converging practitioner-level signals that TDD improves perceived defect detection, modularity, and maintainability at the cost of additional development time, a still-nascent (and currently unsupervised) benefit from generative-AI tooling, and a largely unexplored quality dimension – test-code smells – that may offset production-code gains. We conclude that TDD’s impact on software quality is real, positive on balance, but conditional, instrument-dependent, and incompletely characterized by the current literature, and we outline concrete methodological steps the field must take before the debate can be considered resolved.

Index Terms—Test-Driven Development, TDD, Software Quality, Defect Density, Maintainability, Development Effort, Development Time, Test Coverage, Test Smells, Measurement Validity

I. INTRODUCTION

A. Background and Motivation

Test-Driven Development (TDD) is an agile software engineering practice built around a short, disciplined “red-green-refactor” cycle: a developer first writes a failing automated test that encodes a small unit of desired behavior (red), then writes the minimal implementation code required to make

that test pass (green), and finally refactors the code while keeping all tests passing. Proponents have argued for over two decades that this discipline should systematically improve software quality through three mechanisms: (i) it forces explicit, testable specification of behavior before implementation, reducing ambiguity-driven defects; (ii) it produces a growing regression-test suite as a natural by-product of development, which should catch defects introduced by later changes; and (iii) it incentivizes small, decoupled, more easily testable units of code, which is often assumed to correlate with better internal design quality.

Despite this intuitive case, and despite more than twenty years of controlled experiments, industrial case studies, and systematic reviews, the empirical software engineering community has not converged on a stable answer to whether TDD actually delivers these quality benefits in practice. Early foundational experiments (e.g., George and Williams, 2004; Janzen and Saiedian, 2008) reported measurable quality gains, while other well-controlled studies found no statistically significant difference, or even found TDD underperforming alternative quality-assurance practices such as code inspection. This persistent disagreement motivates the present report: rather than adding a sixth isolated empirical study to the pile, we instead synthesize five recent (2024) studies that, taken together, begin to explain *why* the field disagrees, not merely *whether* TDD works.

B. Research Question

This report addresses:

RQ2: How does Test-Driven Development (TDD) impact software quality?

We decompose “software quality” into five concrete, measurable or perceivable sub-dimensions that recur across the selected literature: (1) overall/general software quality (defect detection, correctness), (2) maintainability (modularity, coupling, cohesion, test-code cleanliness), (3) defect density, (4) development time and effort, and (5) test coverage / test thoroughness.

C. Contribution and Structure

This report’s contribution is not a new primary experiment but a rigorously verified, structured synthesis: every cited paper was independently confirmed to exist (via direct retrieval of its official publisher page, DOI resolution, or

arXiv abstract page) before being included, and no statistic, quote, or finding attributed to a source was invented. The remainder of this report is organized as follows. Section II describes the literature search and screening methodology in detail. Section III presents the full evidence table. Section IV reports detailed findings for each of the five selected studies, structured explicitly around objective, methodology, dataset, experimental design, metrics, statistical analysis, findings, and limitations. Section V synthesizes the findings into a cross-study discussion organized around the five quality sub-dimensions defined above, including a direct point-by-point agreement/contradiction table. Section VI enumerates the research gap in detail. Section VII concludes strictly on the basis of the evidence gathered.

II. LITERATURE REVIEW METHODOLOGY

A. Search Sources

In accordance with the assignment constraints, the search was restricted exclusively to the following five sources: Google Scholar, OpenAlex, ResearchGate, Connected Papers, and Litmaps.

B. Search String

The following Boolean query (and minor lexical variants, e.g., substituting "impact" for "effect") was applied across the above sources:

```
("Test-Driven Development"
OR "TDD") AND ("software
quality" OR "defect density" OR
"maintainability" OR "development
effort" OR "development time" OR
"test coverage")
```

Searches were restricted to publications dated 2024–2026 to ensure the evidence base reflects the most current empirical and review literature, in line with the explicit instruction to prioritize recent studies over the 2004–2023 literature used in earlier drafts of this report.

C. Screening and Verification Process

Candidate papers were screened in two stages. *Stage 1 (relevance screening)*: title and abstract were checked against the inclusion criteria below. *Stage 2 (existence verification)*: for every paper that passed Stage 1, the report's author independently retrieved the paper's official publisher page (or, for preprints, its arXiv abstract page) and cross-checked the title, authorship, publication venue, year, and DOI/arXiv ID against at least one secondary indexing source (e.g., ResearchGate, DOAJ, or a citing paper's reference list). No paper was cited on the basis of memory or inference alone.

Inclusion criteria: (1) the study empirically investigates, qualitatively explores, or systematically reviews TDD's relationship to at least one software-quality-related outcome; (2) the study was published in 2024 or later; (3) the study is retrievable and verifiable through one of the five permitted sources; (4) the study is written in English.

Exclusion criteria: (1) purely opinion or tutorial pieces with no empirical or review component; (2) studies that could not be independently verified to exist; (3) studies published before 2024, except where explicitly retained as background context in Section I.

D. Selected Studies

Five studies satisfied all inclusion and verification criteria and form the evidence base for this report. Table I summarizes their key characteristics; full narrative findings are presented in Section IV.

III. FINDINGS FROM SELECTED STUDIES

A. Evaluating the Impact of TDD on Software Quality Enhancement [1]

Objective. This study set out to evaluate TDD's effectiveness in improving software quality, with explicit sub-goals of assessing its role in early defect detection, code design and maintainability, and developer productivity, and to compare TDD against traditional (non-TDD) development approaches from the practitioner's own perspective.

Methodology. A mixed-methods design combining quantitative surveys and qualitative interviews with practicing software developers. This is a perception-based design: rather than instrumenting real codebases, the authors collected developers' self-reported assessments of TDD's effects.

Dataset. The dataset consists of structured survey responses and semi-structured interview transcripts gathered from software developers with TDD experience; no source code, defect logs, or static-analysis output were collected or analyzed.

Experimental Design. There was no controlled experiment or randomized treatment assignment; instead, the study relies on retrospective comparative analysis, asking respondents to compare their experience of TDD against traditional development methodologies they had also used.

Metrics. All metrics are perceptual: perceived rate/ease of early defect detection, perceived improvement in code design (modularity, coupling), perceived maintainability, and perceived developer productivity.

Statistical Analysis. The quantitative survey component was analyzed descriptively (frequency/percentage of responses per item); the interview component was analyzed thematically to extract recurring qualitative patterns. No inferential hypothesis testing (e.g., t-tests, regression) is reported.

Findings. The survey results indicate that TDD offers substantial perceived benefits in early defect detection, which respondents linked to reduced downstream cost and effort in fixing defects later in the lifecycle. TDD was also reported to encourage improved code design and maintainability, fostering more modular, loosely-coupled code structures. Comparative analysis against traditional methodologies further reinforced TDD's perceived effectiveness at enhancing overall software quality. At the same time, respondents identified concrete adoption barriers: a higher initial time investment required to write tests before code, and difficulty adapting test suites when requirements change frequently.

TABLE I
EVIDENCE TABLE OF SELECTED STUDIES (2024)

Paper	Authors	Year	Study Type	Dataset	Participants	Quality Metrics	Results	Limitations
Evaluating the Impact of TDD on Software Quality Enhancement [1]	Rahman, Saha, Chakraborty, Sujana, Shafi	2024	Mixed-methods (survey + interviews)	Practitioner survey/interview responses	Software developers (self-selected)	Perceived defect detection, design quality, maintainability, productivity	TDD perceived to reduce defects, improve modularity/maintainability; higher perceived effort	Self-report bias; no controlled/objective code metrics
Relevant Information in TDD Experiment Reporting [2]	Uyaguari, Vegas, Dieste et al.	2024	Mixed method: quantitative re-analysis + SMS	FSecure industry experiment data; 18 primary TDD-quality studies	24 professionals (FS experiment); 18 studies in SMS	External quality (QLTY, % assertions passed)	TDD effect sign flips (+21.9 to -12.8 pts) depending on test suite; across 18 studies: 38.8% positive, 11.1% negative, 50% no difference	Single reanalyzed dataset; SMS limited to English, indexed studies
Generative AI for TDD: Preliminary Results [3]	Mock, Melegati, Russo	2024	Exploratory experiment	One programming task (TextFormatter)	5 developers + 1 fully-automated AI run	#tests, #assertions, test/code LOC, time, edge-case coverage	AI-assisted TDD faster but shallower test suites; AI sometimes “fixes” tests instead of bugs	n=5, single task, single LLM version, no inferential stats
Comparative Study on TDD and BDD on Enterprise Delivery Effectiveness [4]	Cui	2024	Qualitative (interviews)	Enterprise practitioner interviews	Developers & project managers	Perceived defect rate, delivery speed, collaboration quality	TDD: enhanced quality, fewer defects, higher time cost; BDD: better communication, requirement-clarity issues	Qualitative only; no code-level metrics; single-author study
TDD Practice and Test Smells – SLR [5]	Marabesi, García-Holgado, García-Peñalvo	2024	Systematic Literature Review	Primary studies from Scopus, WoS, Google Scholar	N/A (review)	Test-smell prevalence, code comprehension, org. context	Org. structure influences testing strategy; smells hurt comprehension; TDD-style/smell link under-researched	Review limited by primary-study quality/heterogeneity; publication bias risk

Limitations. Because every outcome is self-reported rather than independently measured, the study is vulnerable to social-desirability bias (practitioners who have invested effort in learning TDD may be motivated to report it positively) and confirmation bias. The absence of a matched non-TDD control group measured on the same objective metrics (defect counts, complexity, coverage) means the study can establish correlation between TDD experience and perceived quality, but not a causal effect.

B. Relevant Information in TDD Experiment Reporting [2]

Objective. Unlike the other four papers in this evidence base, this study does not ask “does TDD improve quality” directly. Instead, it investigates a prior, meta-methodological question: do the measurement instruments used *within* TDD experiments (test-suite design, code-intervention procedure, and identity of the person performing the measurement) themselves materially affect the reported quality outcome – and can this explain why the TDD literature is so inconsistent?

Methodology. A sequential mixed-method design with two phases. The quantitative phase reanalyzes the “FSecure” (FS) industrial TDD experiment using Bland-Altman agreement plots – a statistical technique borrowed from medical measurement-instrument comparison – to compare quality scores obtained under different measurement conditions. The qualitative phase is a systematic mapping study (SMS) following Kitchenham and Charters’ guidelines.

Dataset. The quantitative reanalysis uses the original FS experiment dataset: 24 professional developers across three company sites, who each completed two tasks (MarsRover API

and Bowling Score Keeper) under TDD and an incremental-test-last (ITL) control condition. The SMS dataset comprises 18 primary TDD-external-quality experiments, filtered down from an initial pool of 1,922 candidate publications retrieved from IEEE Xplore, ACM Digital Library, SpringerLink, and ScienceDirect.

Experimental Design. For the quantitative phase, the same delivered source code was re-measured multiple times under controlled variations of three “operationalization components”: (i) test-suite generation technique (ad hoc vs. equivalence partitioning), (ii) code-intervention type (no intervention, syntactic intervention, semantic intervention), and (iii) measurer identity (two independent researchers measured the same code).

Metrics. The response variable throughout is external quality (QLTY), operationalized as the percentage of acceptance-test assertions passed by the code a subject delivered.

Statistical Analysis. Linear regression models ($QLTY = \text{Treatment} + \epsilon$) were fit separately for each measurement condition, reporting regression coefficients, standard errors, and significance levels; Bland-Altman plots quantified the mean bias and 95% limits of agreement between paired measurements taken under different conditions.

Findings. The single most consequential finding is that the estimated TDD treatment effect changes sign depending on which test suite is used to measure the same delivered code: using an ad hoc test suite, TDD showed a statistically significant *positive* effect of +21.93 quality points ($p < 0.05$); using an equivalence-partitioning test suite on the identical

code, TDD showed a statistically significant *negative* effect of -12.84 points ($p < 0.05$). Code-intervention type had a smaller but still non-trivial effect (the TDD effect ranged from $+49.52$ points under "no intervention" to $+16.15$ points under "semantic intervention"). By contrast, measurer identity had a negligible effect (a difference of only 0.72 points between two independent measurers), indicating that human rater variability is *not* a major source of the field's inconsistent results. The companion SMS of 18 published TDD-quality experiments found that TDD improved quality in 38.8% of studies, reduced it in 11.1%, and showed no significant difference in 50.0% – and, critically, that none of the 18 studies fully reported the test cases used to measure quality, undermining replicability.

Limitations. The quantitative claim about measurement sensitivity rests on a single reanalyzed industrial dataset (FS experiment); while internally rigorous, it is not yet known how far this specific pattern generalizes to other codebases, languages, or task types. The SMS is restricted to English-language, database-indexed studies and may miss grey literature or non-indexed industrial reports.

C. Generative AI for Test Driven Development: Preliminary Results [3]

Objective. To explore whether generative AI (specifically, OpenAI's ChatGPT) can automate or assist the TDD red-green-refactor cycle, to define interaction patterns between human developers and the AI, and to assess how different levels of AI automation affect the quality and completeness of the resulting test suite and code.

Methodology. An exploratory mixed quantitative/qualitative experiment. The authors first engineered a structured prompting workflow around the OpenAI API (model `gpt-3.5-turbo-16k`) that incrementally drives the TDD cycle, then defined three human-AI collaboration patterns – *collaborative* (developer writes/edits tests, AI generates production code), *fully-automated* (AI performs both roles, developer only verifies at refactor time), and *non-automated* (baseline, no AI involvement) – and implemented the workflow as a Python automation script.

Dataset / Participants. Five practitioner developers located in two different countries, with varying levels of TDD and Python experience, each completed a single programming task: implementing a `TextFormatter` class that horizontally centers words within a line, using Test-First/Assertion-First TDD. Three participants used the collaborative pattern; two used the non-automated (no-AI) baseline. One additional fully-automated run (labeled F1) was performed by the authors themselves as a reference point.

Experimental Design. All participants received an identical task specification and a 40-minute time limit. Screen recordings and interaction logs were captured for all AI-assisted sessions, allowing the authors to inspect not just final outcomes but the step-by-step iteration history.

Metrics. Number of test functions, number of assertions, test lines of code (LOC), production code LOC, time to complete the task, and number of TDD iterations were recorded

quantitatively; test/code quality (edge-case coverage, correctness, cleanliness) was assessed qualitatively via manual code and log inspection, supplemented by a short post-task feedback questionnaire.

Statistical Analysis. Given the very small sample ($n=5$ participants plus one AI-only run), the authors report purely descriptive comparisons in a single results table; no significance testing, confidence intervals, or effect-size statistics were computed or claimed.

Findings. All six sessions (five human plus the fully-automated F1) successfully completed the task within the time limit. The fully-automated AI run was markedly faster (12 minutes, 8 iterations) than any human participant (30–40 minutes), but produced the shallowest test suite: only one test function with three assertions, missing an entire valid-input case and no edge cases. Human participants using the collaborative pattern generally produced richer test suites with more edge-case coverage (e.g., one participant added tests for words longer than the line width and empty-string input), though quality varied by individual – one participant's final code remained incorrect even after AI-assisted "fixes." Critically, log inspection revealed a specific and quality-relevant failure mode: when a test failed, the AI sometimes proposed changes that altered the *test's* expected value or variable assignment to make the test pass, rather than correcting the underlying bug in the production code – a change that superficially resolves the red-green cycle without truly fixing the defect. Participant feedback was generally positive when the AI's behavior matched the developer's own workflow expectations, but one participant reported frustration when the automation script failed to parse function-style (rather than class-style) code from the AI's response.

Limitations. The sample is very small (five developers, one task, one LLM version), non-random, and drawn informally; no statistical inference is possible or claimed. The single task (a text-formatting exercise) is far simpler than typical industrial TDD scenarios, and the specific LLM version used (`gpt-3.5-turbo-16k`) is now several generations behind current models, which may limit the currency of the quality findings.

D. A Comparative Study on TDD and BDD on Enterprise Software Delivery Effectiveness [4]

Objective. To compare the impact of Test-Driven Development (TDD) and Behavior-Driven Development (BDD) on enterprise software delivery effectiveness, specifically examining delivery speed, software quality, and team collaboration, in order to help enterprises select the practice best aligned with their project type and stakeholder needs.

Methodology. A qualitative research design based on in-depth, semi-structured interviews with developers and project managers at enterprises that had adopted either TDD or BDD as their primary testing-driven practice.

Dataset. Interview transcripts collected from enterprise practitioners (developers and project managers) working at

organizations practicing TDD or BDD; no source code, defect-tracking data, or delivery-pipeline metrics were directly analyzed – all data is practitioner-reported through interviews.

Experimental Design. There is no controlled comparison or randomized assignment; the design is a between-groups qualitative comparison, contrasting the lived experience of TDD-practicing interviewees against BDD-practicing interviewees.

Metrics. Thematically coded, qualitative constructs: perceived defect rate/software quality, perceived delivery speed, and perceived cross-functional team-collaboration effectiveness.

Statistical Analysis. As a purely qualitative study, thematic/content analysis of interview transcripts was used to identify recurring patterns; no quantitative statistical testing is reported or appropriate for this design.

Findings. The findings reveal that TDD and BDD have distinct, largely complementary rather than strictly competing effects. TDD's emphasis on early, iterative testing was consistently reported by interviewees to lead to enhanced code quality and fewer defects reaching later stages of delivery. In contrast, BDD's focus on human-readable behavior specifications was reported to improve cross-functional communication by directly involving business stakeholders in defining expected behavior. However, each practice carries its own cost: TDD was associated with a higher initial time investment (writing tests before any implementation exists), while BDD was associated with challenges in achieving requirement clarity when behavior specifications were ambiguously written. The author frames these differences as guidance for practice selection based on project type and stakeholder involvement needs, rather than declaring one practice categorically superior.

Limitations. The entirely qualitative, interview-based design limits statistical generalizability and cannot quantify the magnitude of TDD's (or BDD's) quality benefit in the way a controlled experiment could. Interview-based data is also subject to recall bias, social-desirability bias, and interviewer/coder subjectivity in theme extraction. As a single-author study, there is an elevated risk of unchecked interpretive bias during qualitative coding, since no second coder or inter-rater reliability check is reported.

E. Exploring the Connection between the TDD Practice and Test Smells – A Systematic Literature Review [5]

Objective. To systematically review the literature at the intersection of TDD practice and *test smells*/anti-patterns (poor design or implementation choices in test code itself, analogous to "code smells" in production code), and to understand how organizational and practice-related factors influence the introduction of such smells – a quality dimension essentially absent from the other four studies in this evidence base, which focus on production-code or perceived quality.

Methodology. A systematic literature review (SLR) conducted according to established SLR guidelines, searching Scopus, Web of Science, and Google Scholar for primary studies addressing TDD's external/internal quality, productivity, test smells, and code comprehension.

Dataset. The "dataset" is the corpus of peer-reviewed primary studies retrieved and screened through the three databases; the review explicitly notes that TDD studies in this corpus cluster into four focus areas: TDD adoption, quality/productivity, quality, and practice, with test smells and anti-patterns historically treated as an "isolated" topic rarely connected to TDD practice itself.

Experimental Design. As a secondary study, there is no primary experimental design; instead, the review applies explicit inclusion/exclusion criteria and a structured data-extraction protocol across the screened corpus, following recognized SLR methodology (analogous to the Kitchenham and Charters approach used in [2]'s SMS component).

Metrics. Qualitative/bibliometric: categorization of study focus area, frequency and type of reported test smells / TDD anti-patterns, and thematic linkage between organizational context and testing strategy, reported test-code comprehension, and TDD style (e.g., inside-out vs. outside-in / "TDD double-loop").

Statistical Analysis. Descriptive and bibliometric analysis (frequency counts, focus-area correlation charts across the reviewed corpus) rather than quantitative effect-size meta-analysis, consistent with the qualitative, review-oriented nature of an SLR on a topic where primary studies use heterogeneous outcome measures.

Findings. The review finds that organizational structure has a measurable influence on testing strategy and, by extension, on the propensity to introduce test smells into a codebase's test suite. It further finds a persistent conceptual conflation in the literature: test smells and TDD anti-patterns are frequently used interchangeably by authors despite representing distinct constructs, which the review argues muddies cross-study comparison. Critically for RQ2, the review confirms that test smells negatively impact test-code comprehension – meaning that even when TDD improves production-code maintainability (as reported in [1]), it may simultaneously be degrading the maintainability of the test suite itself if smells accumulate unchecked. Finally, the review identifies that the specific *style* of TDD practiced (inside-out, which starts from a unit and builds outward, versus outside-in/"double-loop," which starts from an acceptance test and works inward) and its relationship to test-smell generation is frequently overlooked entirely by the primary literature, representing an open sub-question.

Limitations. As a secondary study, the review's conclusions are only as strong as the quality and consistency of the underlying primary studies it screened; heterogeneous operationalizations of "quality" and "test smell" across the primary literature limit the strength of cross-study comparison. The review's own peer-review record (publicly available alongside the MDPI publication) notes that its research-gap framing initially lacked focus, which the authors addressed but which nonetheless signals some structural limitation acknowledged by independent reviewers. There is also a standard risk of publication bias inherent to any SLR relying on indexed, published literature.

IV. DISCUSSION

This section synthesizes the five studies along the five software-quality sub-dimensions introduced in Section I.C, and then presents an explicit cross-study agreement/contradiction matrix.

A. Software Quality (General)

At the perceptual level, both [1] and [4] independently report that practitioners associate TDD with improved overall quality and reduced defects reaching later development stages – a striking convergence given the two studies used entirely different populations (Bangladeshi developers surveyed/interviewed by Rahman et al. vs. enterprise developers/PMs interviewed by Cui) and entirely different qualitative methods. At the measured level, [2]’s SMS of 18 primary experiments shows a plurality but not a majority of studies (38.8%) find a positive quality effect, with fully half (50.0%) finding no significant difference. Taken together, this suggests a real gap between practitioner *perception* of TDD’s quality benefit and what is reliably *measured* in controlled experiments – consistent with the possibility that perceived benefits are partly attributable to increased overall testing effort and attention (a Hawthorne-like effect) rather than the TDD discipline per se.

B. Maintainability

[1] reports that TDD fosters modular, loosely-coupled production code, a claim consistent with two decades of prior literature on TDD’s design-quality effects. However, [5] introduces an important asymmetry: maintainability gains in *production* code do not guarantee maintainability of the *test* code, and if test smells accumulate, overall maintainability of the codebase-as-a-whole may not improve as much as production-code-only assessments would suggest. This is, to our knowledge, a genuinely novel synthesis point not made explicit in the individual papers themselves: no primary study in this evidence base jointly measures both production-code and test-code maintainability for the same subjects, meaning the net maintainability effect of TDD remains an open empirical question.

C. Defect Density

[4] and [1] both report perceived defect-density reductions attributable to TDD. [2], however, offers the most methodologically rigorous evidence in this evidence base and shows that a defect/quality proxy result for the *identical* delivered code can swing from a large, statistically significant improvement to a large, statistically significant decline purely as a function of test-suite design choice. This is the single most important caveat for RQ2: any defect-density claim drawn from the TDD literature – including the perceptual claims in [1] and [4] – should be read with the understanding that the underlying “ground truth” defect measurement itself is not a fixed, objective quantity in this research area, but a construct highly sensitive to instrumentation choices that are rarely reported in enough detail to assess (as [2]’s SMS also found: none of the 18 reviewed experiments fully reported their test cases).

D. Development Time and Effort

This is the dimension with the strongest cross-study agreement. [1] and [4] independently report a higher initial time investment for TDD compared to traditional/non-TDD approaches, consistent with the broader TDD literature’s long-standing “quality costs time” trade-off. [3] adds a new wrinkle specific to 2024: generative-AI assistance can substantially compress this time cost (the fully-automated AI run completed in 12 minutes versus 30–40 minutes for human participants), but appears to do so by trading away test thoroughness – suggesting the classic time/quality trade-off has not been eliminated by AI tooling, merely reshaped into a new form (time saved vs. test-suite depth, rather than time spent vs. defect rate).

E. Test Coverage

[3] provides the only direct, comparative evidence on test coverage in this evidence base: collaborative human-AI TDD sessions produced measurably richer test suites (more assertions, explicit edge-case handling) than the fully-automated, AI-only session, which produced a single shallow test function. This suggests that, at least with current-generation tooling, human oversight remains a necessary ingredient for test-coverage quality even as AI accelerates the mechanical parts of the TDD loop – an important, practically actionable finding for teams considering AI-assisted TDD adoption.

F. Cross-Study Agreement and Contradiction Matrix

TABLE II
CROSS-STUDY AGREEMENT / CONTRADICTION SUMMARY

Dimension	Cross-Study Pattern
Software Quality	Agreement at perceptual level ([1], [4]); contradicted/qualified at measured level ([2]: only 38.8% of experiments show a positive effect)
Maintainability	Partial agreement (production code: [1]); unaddressed counter-risk identified (test code: [5]) – no study measures both jointly
Defect Density	Perceptual agreement ([1], [4]); direct methodological contradiction demonstrated within a single dataset ([2])
Development Time/Effort	Strong agreement across all applicable studies ([1], [4]); reshaped but not resolved by AI tooling ([3])
Test Coverage	Only addressed by one study ([3]); no contradiction, but no independent replication either

The overall pattern that emerges is that the five studies do *not* straightforwardly contradict one another at the level of raw claims – most agree TDD has some positive association with quality-related outcomes. Where they genuinely diverge is in *confidence and mechanism*: perceptual/qualitative studies ([1], [4]) report consistent practitioner-level benefit, while the one study that interrogates measurement validity directly ([2]) shows that this benefit is far less certain and far more instrument-dependent than perceptual studies would suggest, and the two more exploratory/novel-angle studies ([3], [5]) each surface a previously under-examined risk (AI-induced shallow testing; test-code smells) that could partially offset the benefits reported elsewhere.

V. RESEARCH GAP

Based strictly on the five studies synthesized above, the following unresolved issues, inconsistent findings, industrial limitations, and future-work directions are identified.

A. Unresolved Issues

- (1) **No standardized quality-measurement protocol exists for TDD experiments.** [2] demonstrates that the field’s most basic empirical instrument – the acceptance-test suite used to score “quality” – is neither standardized nor even consistently reported (none of 18 reviewed experiments fully disclosed their test cases), making cross-study comparison and meta-analysis unreliable by construction, not merely by chance sampling variation.
- (2) **Production-code and test-code quality are never jointly measured.** [1] reports production-code maintainability gains while [5] reports a distinct risk to test-code maintainability via test smells, but no study in this evidence base (or, per the SLR’s own account of the broader literature) measures both dimensions simultaneously for the same subjects, leaving the *net* maintainability effect of TDD unknown.
- (3) **Perceptual and measured quality diverge without reconciliation.** [1] and [4] report strong practitioner-perceived benefit, while [2]’s SMS shows only a minority (38.8%) of directly measured experiments find a significant positive effect; no study in this set investigates *why* this perception-measurement gap exists (e.g., attention/effort effects, selection bias among TDD adopters, or genuine measurement insensitivity).

B. Inconsistent Findings

The clearest documented inconsistency is internal to [2] itself: identical delivered source code, measured with two different (both defensible) acceptance-test-suite generation techniques, yields a statistically significant positive TDD effect under one technique and a statistically significant negative effect under the other. This is not merely “mixed results across different studies” (which could be attributed to differing populations, tasks, or contexts) but a demonstrated instrument-driven sign flip *within one dataset*, which is a substantially stronger and more concerning form of inconsistency than typically acknowledged in TDD meta-discussions.

C. Industrial Limitations

Three of the five studies rely on data sources with limited industrial rigor: [1] and [4] use self-reported perceptual data (survey/interview) rather than instrumented production metrics, and [3]’s experiment, while promising as an early look at AI-assisted TDD, involves only five participants on a single, small, non-industrial programming task. Only [2]’s quantitative component draws on genuine industrial data (the FSecure experiment, 24 professionals across three company sites), underscoring how thin the industrial-grade evidence base for RQ2 remains even in 2024.

D. Future Work

Building directly on the gaps above, we recommend: (1) development and adoption of a standardized, pre-registered external-quality measurement protocol for TDD experiments, explicitly requiring publication of the test suite used, following the transparency recommendations implicit in [2]’s findings; (2) experimental designs that jointly measure production-code and test-code quality (including test-smell density) within the same study, directly connecting the findings of [1] and [5]; (3) larger-sample, industrial-scale, multi-task replications of AI-assisted TDD building on the exploratory design of [3], ideally with current-generation LLMs and with statistical power sufficient for hypothesis testing; and (4) mixed-methods studies that explicitly triangulate perceptual survey/interview data (as in [1], [4]) against objective, instrumented code metrics for the *same* subjects, to directly test whether the perception-measurement gap identified in Section VI.A(3) reflects genuine measurement insensitivity or a real practitioner-perception bias.

VI. CONCLUSION

Based strictly on the evidence gathered from five independently verified 2024 studies, this report concludes that TDD is associated with a positive but conditional, and demonstrably measurement-sensitive, effect on software quality. At the practitioner-perception level, evidence from a mixed-methods survey/interview study [1] and a qualitative enterprise-interview study [4] consistently favors TDD for early defect detection and maintainability, at the acknowledged cost of higher initial development time. However, the only study in this evidence base to directly interrogate measurement validity [2] shows that this positive effect is not robust to how quality is operationalized, and can reverse sign entirely within a single dataset depending on test-suite design – a finding that should materially temper confidence in any single-instrument TDD-quality claim, including the perceptual claims reported elsewhere in this same evidence base. Emerging 2024 evidence on generative-AI-assisted TDD [3] suggests that automation can meaningfully reduce TDD’s time cost, but at a currently observed cost to test thoroughness absent adequate developer oversight. Finally, systematic review evidence [5] establishes that TDD’s effect on *test-code* quality – via the introduction of test smells – is a distinct and still substantially under-researched quality dimension that no study in this evidence base measures jointly with production-code quality. No claim beyond this conditional, instrument-sensitive, and dimension-incomplete positive effect is supported by the evidence synthesized in this report; a confident, unqualified verdict on “does TDD improve software quality” is not currently defensible given the state of the 2024 literature reviewed here.

REFERENCES

- [1] M. S. Rahman, A. K. Saha, U. Chakraborty, H. T. Sujana, and S. M. A. Shafi, “Evaluating the impact of Test-Driven Development on Software Quality Enhancement,” *International Journal of Mathematical Sciences and Computing (IJMSC)*, vol. 10, no. 3, pp. 51–76, 2024, doi: 10.5815/ijmsc.2024.03.05.

- [2] F. Uyaguari, S. Vegas, O. Dieste, et al., “Relevant Information in TDD Experiment Reporting,” *arXiv preprint*, arXiv:2406.06405, 2024.
- [3] M. Mock, J. Melegati, and B. Russo, “Generative AI for Test Driven Development: Preliminary Results,” *arXiv preprint*, arXiv:2405.10849, 2024.
- [4] J. Cui, “A Comparative Study on the Impact of Test-Driven Development (TDD) and Behavior-Driven Development (BDD) on Enterprise Software Delivery Effectiveness,” *arXiv preprint*, arXiv:2411.04141, 2024.
- [5] M. Marabesi, A. García-Holgado, and F. J. García-Peñalvo, “Exploring the Connection between the TDD Practice and Test Smells—A Systematic Literature Review,” *Computers*, vol. 13, no. 3, p. 79, 2024, doi: 10.3390/computers13030079.